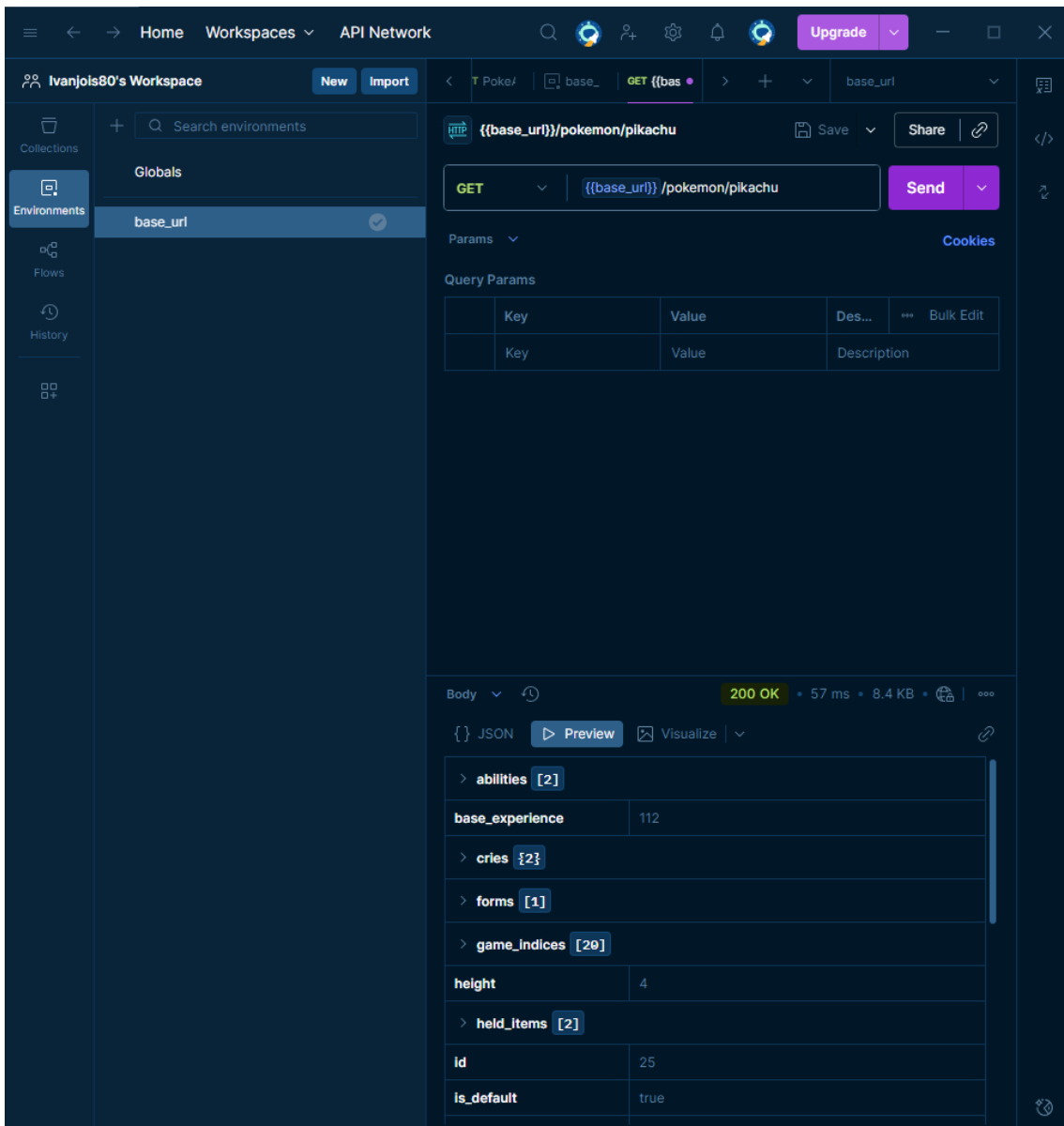


Ejercicios paso a paso con Postman

Ejercicio 0 — Preparación

1. Crea la colección PokeAPI 1DAM.
2. Crea el entorno con la variable `base_url`.
3. Usa URLs del tipo:
4. `{{base_url}}/pokemon/pikachu`

 **Entregable:** Captura del entorno y colección creados.



The screenshot shows the Postman interface with a workspace named 'Ivanjois80's Workspace'. The 'Environments' tab is active, showing a global environment named 'base_url'. A GET request is defined with the URL `{{base_url}}/pokemon/pikachu`. The response is a 200 OK status with a response time of 57 ms and a body size of 8.4 KB. The response body is displayed in JSON format, showing details for the Pokemon Pikachu.

Key	Value	Description
abilities	[2]	
base_experience	112	
cries	{2}	
forms	[1]	
game_indices	[20]	
height	4	
held_items	[2]	
id	25	
is_default	true	

✿ Ejercicio 1 — GET básico

- Método: **GET**
- URL: `{{base_url}}/pokemon/pikachu`
- Observa:
 - Código de estado (¿200 OK?).
 - Cabecera Content-Type.
 - Campos principales (name, height, weight, types...).

📷 **Entregable:** Captura de la respuesta y breve explicación.

The screenshot shows the API Network interface with a GET request to `{{base_url}}/pokemon/pikachu` successfully executed. The response status is 200 OK, with a response time of 57 ms and a size of 8.4 KB. The response body is displayed in JSON format, showing the details of the Pokémon Pikachu.

Query Params

Key	Value	Description
Key	Value	Description

Body

```
{
  "abilities": [
    {
      "ability": "static",
      "is_hidden": false
    },
    {
      "ability": "lightning_rod",
      "is_hidden": false
    },
    {
      "ability": "voltage",
      "is_hidden": true
    }
  ],
  "base_experience": 112,
  "cries": {
    "c1": "pika",
    "c2": "pika",
    "c3": "pika"
  },
  "forms": [
    {
      "form": "Pikachu",
      "is_default": true
    }
  ],
  "game_indices": [
    {
      "game_index": 1,
      "is_default": true
    },
    {
      "game_index": 2,
      "is_default": true
    },
    {
      "game_index": 3,
      "is_default": true
    },
    {
      "game_index": 4,
      "is_default": true
    },
    {
      "game_index": 5,
      "is_default": true
    },
    {
      "game_index": 6,
      "is_default": true
    },
    {
      "game_index": 7,
      "is_default": true
    },
    {
      "game_index": 8,
      "is_default": true
    },
    {
      "game_index": 9,
      "is_default": true
    },
    {
      "game_index": 10,
      "is_default": true
    },
    {
      "game_index": 11,
      "is_default": true
    },
    {
      "game_index": 12,
      "is_default": true
    },
    {
      "game_index": 13,
      "is_default": true
    },
    {
      "game_index": 14,
      "is_default": true
    },
    {
      "game_index": 15,
      "is_default": true
    },
    {
      "game_index": 16,
      "is_default": true
    },
    {
      "game_index": 17,
      "is_default": true
    },
    {
      "game_index": 18,
      "is_default": true
    },
    {
      "game_index": 19,
      "is_default": true
    },
    {
      "game_index": 20,
      "is_default": true
    }
  ],
  "height": 4,
  "held_items": [
    {
      "item": "light_bulb",
      "is_default": true
    }
  ],
  "id": 25,
  "is_default": true
}
```

✿ Ejercicio 2 — Parámetros dinámicos

- Cambia pikachu por otros Pokémon: charmander, bulbasaur, gengar
- Compara campos (height, weight, types, abilities).

 **Entregable:** Tabla comparativa con al menos 3 Pokémon.

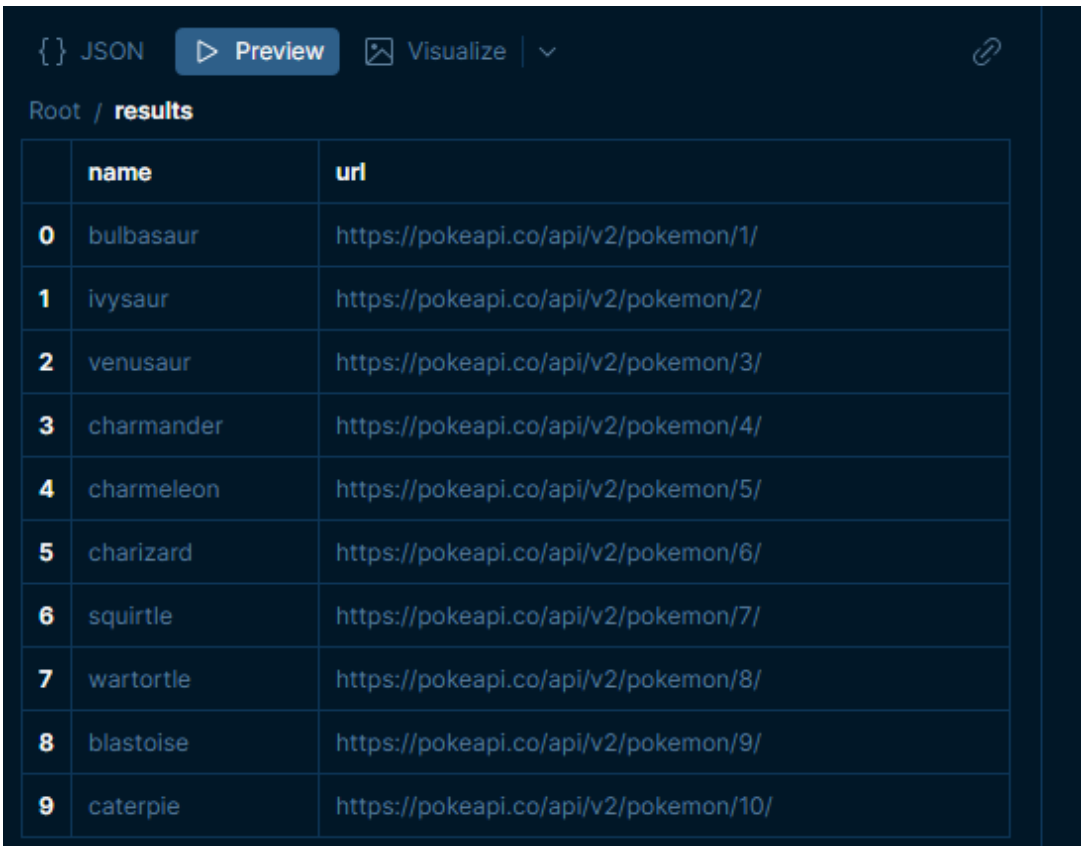
	height	weight	types	abilities
Charmander	6	85	fire	Solar-power
Bulbasaur	7	69	Grass/poison	Chlorophyll
Gengar	15	405	Ghost/poison	Cursed-body

✿ Ejercicio 3 — Listado con paginación

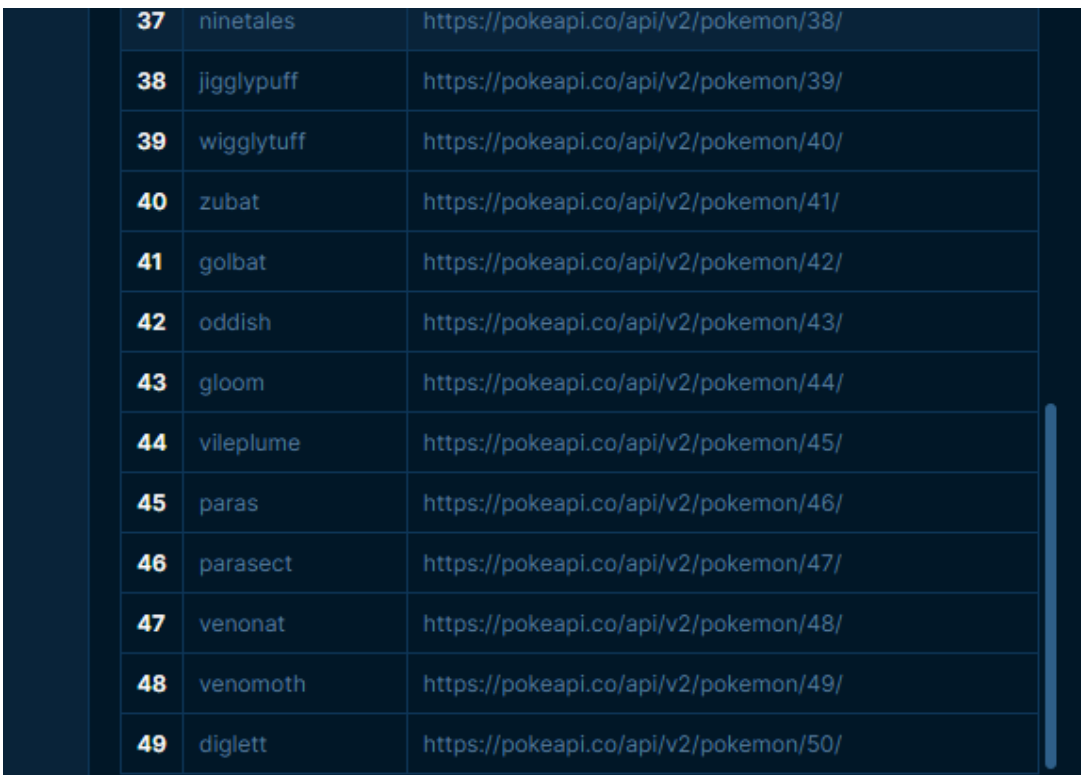
- URL: `{{base_url}}/pokemon?limit=10`
- Observa:
 - Campo results (array).
 - Campos de cada elemento (name, url).
 - Parámetros limit y offset.

🔄 Prueba con limit=50 y observa la diferencia.

📷 **Entregable:** Captura y breve explicación.



	name	url
0	bulbasaur	https://pokeapi.co/api/v2/pokemon/1/
1	ivysaur	https://pokeapi.co/api/v2/pokemon/2/
2	venusaur	https://pokeapi.co/api/v2/pokemon/3/
3	charmander	https://pokeapi.co/api/v2/pokemon/4/
4	charmeleon	https://pokeapi.co/api/v2/pokemon/5/
5	charizard	https://pokeapi.co/api/v2/pokemon/6/
6	squirtle	https://pokeapi.co/api/v2/pokemon/7/
7	wartortle	https://pokeapi.co/api/v2/pokemon/8/
8	blastoise	https://pokeapi.co/api/v2/pokemon/9/
9	caterpie	https://pokeapi.co/api/v2/pokemon/10/



37	ninetales	https://pokeapi.co/api/v2/pokemon/38/
38	jigglypuff	https://pokeapi.co/api/v2/pokemon/39/
39	wigglytuff	https://pokeapi.co/api/v2/pokemon/40/
40	zubat	https://pokeapi.co/api/v2/pokemon/41/
41	golbat	https://pokeapi.co/api/v2/pokemon/42/
42	oddish	https://pokeapi.co/api/v2/pokemon/43/
43	gloom	https://pokeapi.co/api/v2/pokemon/44/
44	vileplume	https://pokeapi.co/api/v2/pokemon/45/
45	paras	https://pokeapi.co/api/v2/pokemon/46/
46	parasect	https://pokeapi.co/api/v2/pokemon/47/
47	venonat	https://pokeapi.co/api/v2/pokemon/48/
48	venomoth	https://pokeapi.co/api/v2/pokemon/49/
49	diglett	https://pokeapi.co/api/v2/pokemon/50/

Me parece como cuando en java recorres un array con un for para ir mostrando lo que contiene.

🌸 Ejercicio 4 — Recursos relacionados

- `{{base_url}}/pokemon-species/25`
- `{{base_url}}/type/electric`
- `{{base_url}}/ability/overgrow`

🧠 Preguntas:

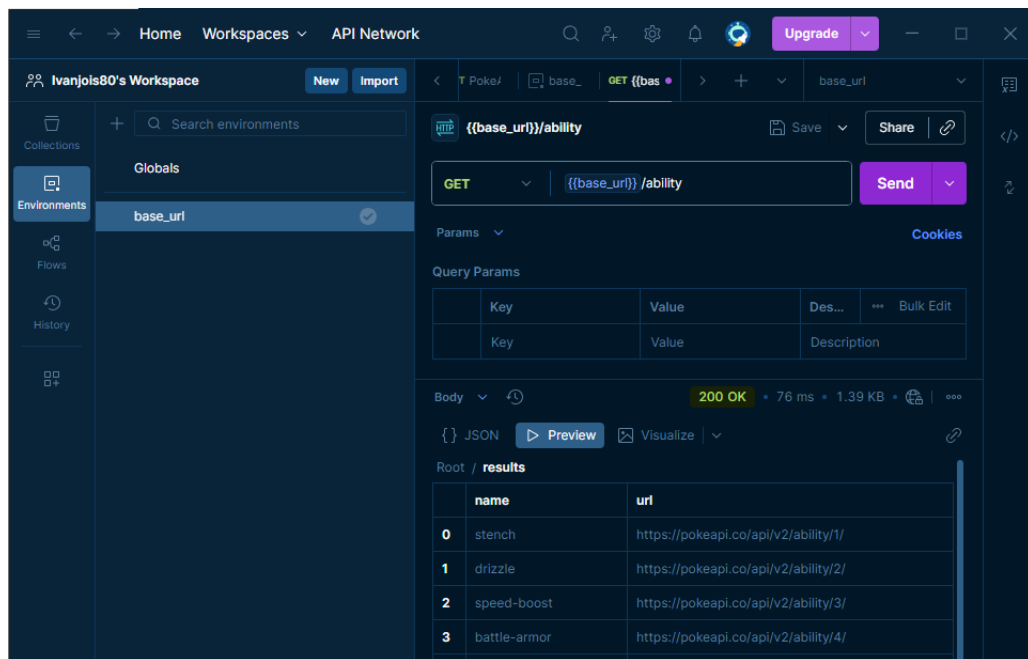
- ¿Qué tipo de información devuelve cada uno?
- ¿Qué relación hay entre `/pokemon` y `/type` o `/ability`?

The screenshot shows the API Network client interface. The left sidebar displays the workspace structure with 'base_url' selected under 'Environments'. The main panel shows a GET request to `{{base_url}}/pokemon`. The response is a 200 OK status with a JSON body. The JSON body contains the following data:

```
{
  "count": 1328,
  "next": "https://pokeapi.co/api/v2/pokemon?offset=20&limit=20",
  "previous": null,
  "results": [
    {
      "name": "bulbasaur",
      "url": "https://pokeapi.co/api/v2/pokemon/1/"
    },
    {
      "name": "ivysaur",
      "url": "https://pokeapi.co/api/v2/pokemon/2/"
    },
    ...
  ]
}
```

The screenshot shows the API Network client interface. The left sidebar displays the workspace structure with 'base_url' selected under 'Environments'. The main panel shows a GET request to `{{base_url}}/type`. The response is a 200 OK status with a JSON body. The JSON body contains the following data:

```
{
  "count": 18,
  "next": "https://pokeapi.co/api/v2/type?offset=4&limit=4",
  "previous": null,
  "results": [
    {
      "name": "flying",
      "url": "https://pokeapi.co/api/v2/type/3/"
    },
    {
      "name": "poison",
      "url": "https://pokeapi.co/api/v2/type/4/"
    },
    {
      "name": "ground",
      "url": "https://pokeapi.co/api/v2/type/5/"
    },
    ...
  ]
}
```

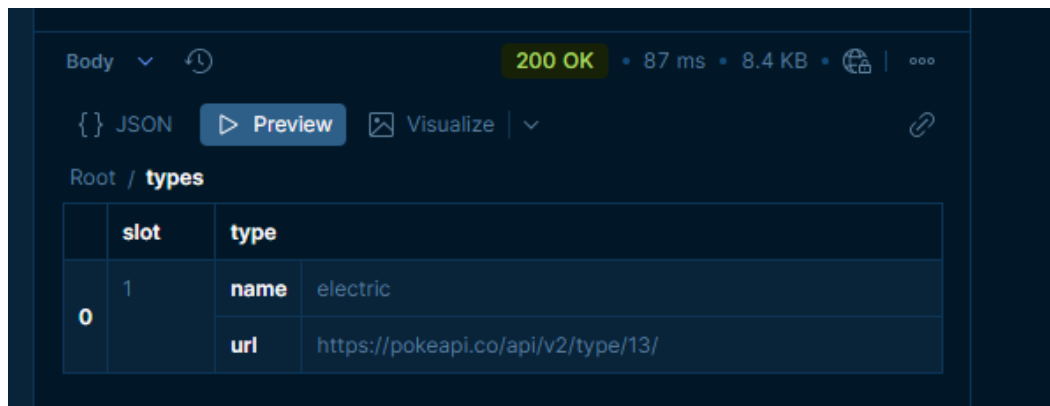


📷 **Entregable:** Esquema Pokémon → Tipo → Habilidad.

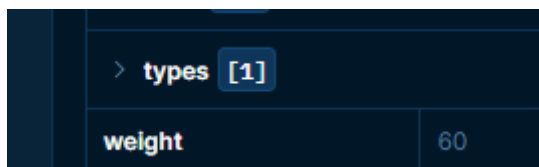
✖ Ejercicio 5 — Campos anidados

En la respuesta de Pikachu:

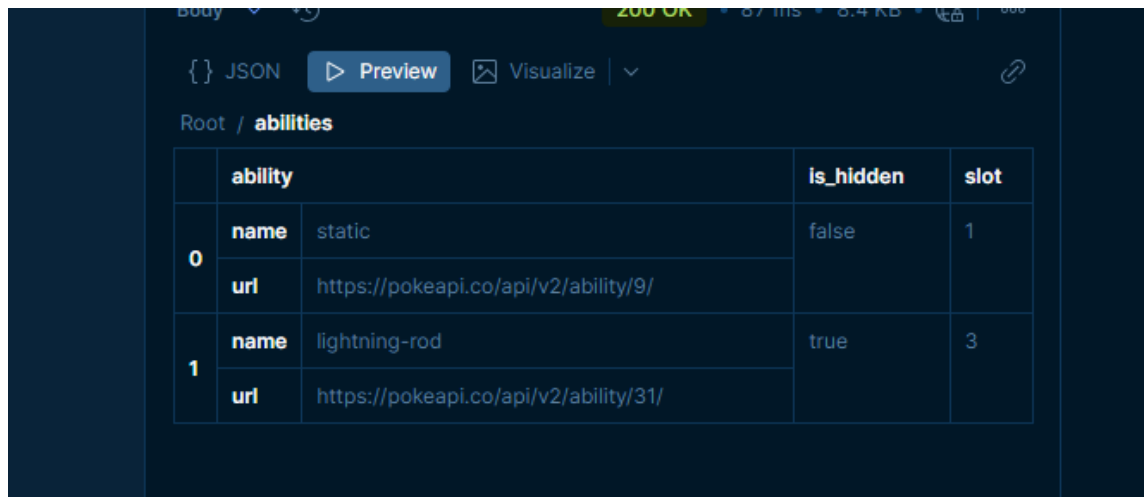
- Primer tipo → types[0].type.name



- Peso → weight



- Primera habilidad → abilities[0].ability.name



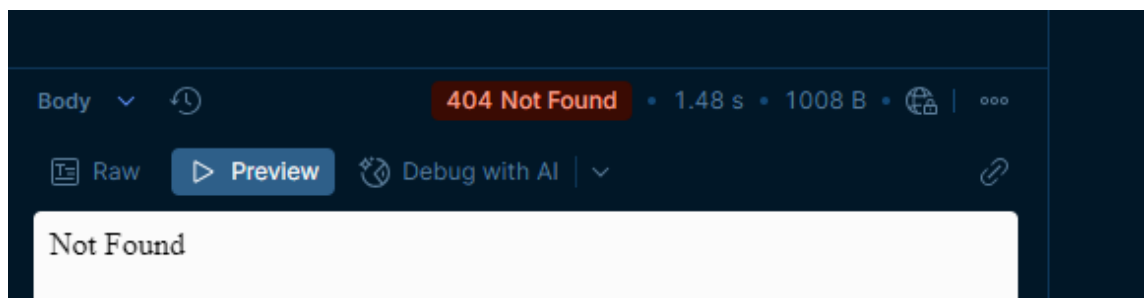
	ability	is_hidden	slot
0	name: static	false	1
	url: https://pokeapi.co/api/v2/ability/9/		
1	name: lightning-rod	true	3
	url: https://pokeapi.co/api/v2/ability/31/		

 **Entregable:** Lista de campos y valores encontrados.

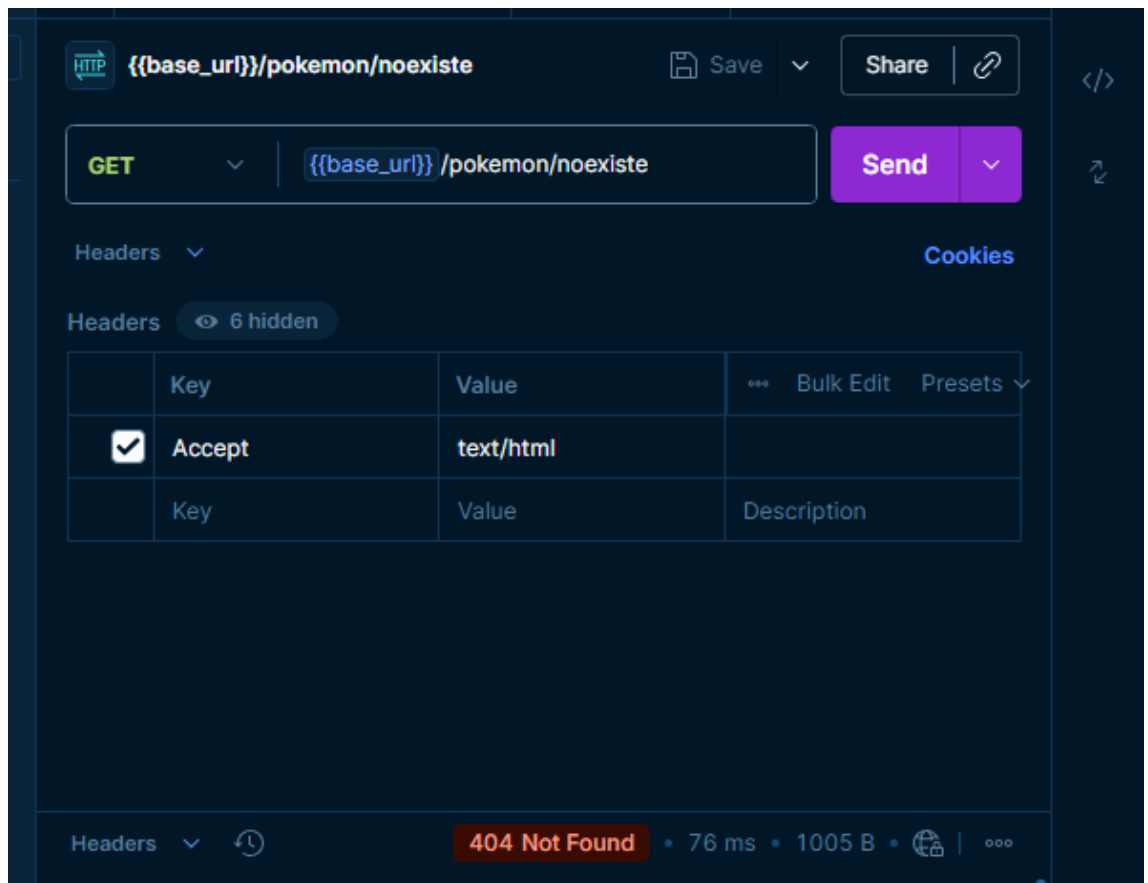
Ejercicio 6 — Errores intencionados (status codes)

- Prueba `{{base_url}}/pokemon/noexiste` → ¿qué devuelve?

Devuelve el error 404 Not Found no encontrado



- Cambia cabecera Accept a text/html. ¿Cambia algo?
No cambia nada porque sigue sin encontrar nada independientemente del formato de los datos que buscamos.

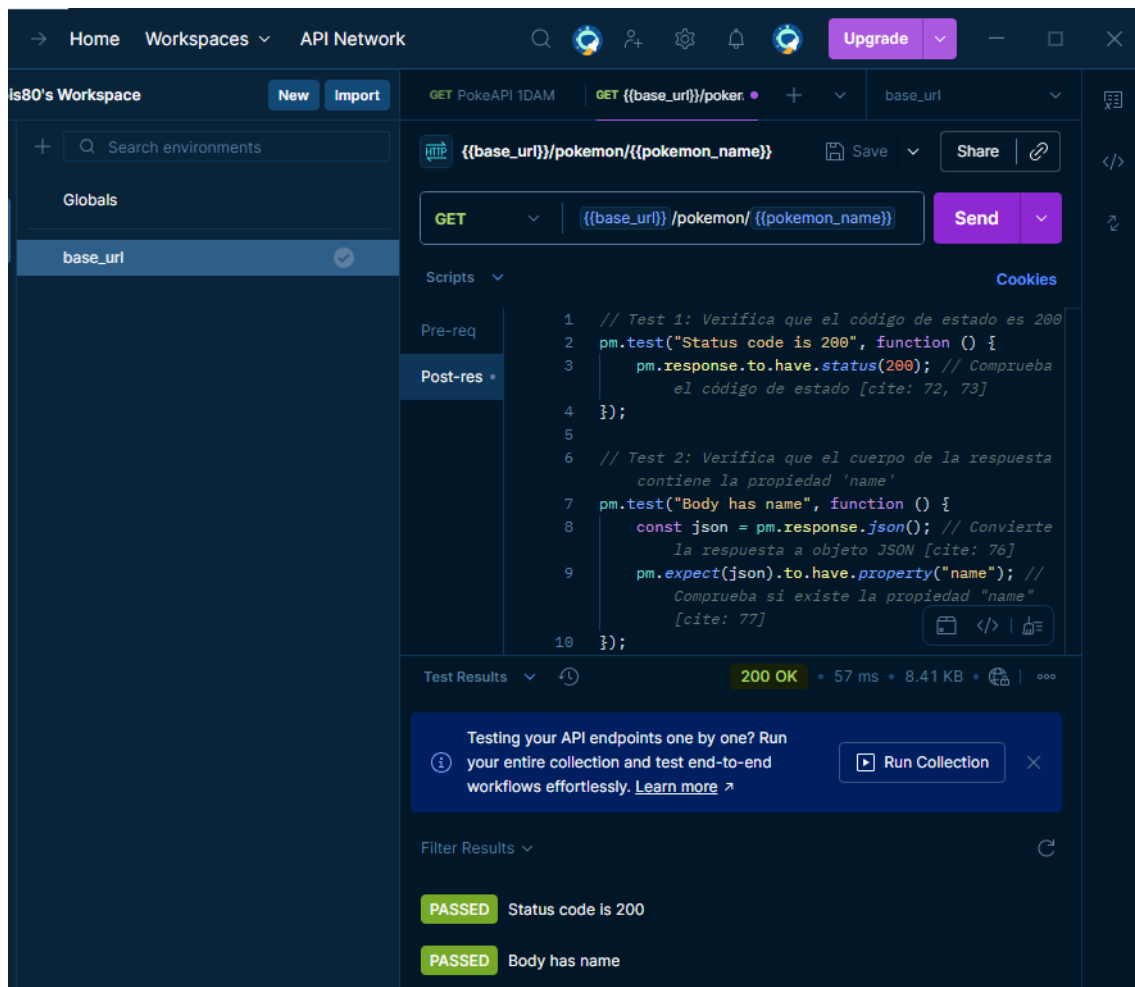


📷 **Entregable:** Tabla con código, significado y explicación.

🌸 Ejercicio 7 — Variables y tests en Postman

1. Crea una variable `pokemon_name` con valor `pikachu`.
2. URL: `{{base_url}}/pokemon/{{pokemon_name}}`
3. En la pestaña **Tests**, escribe:
4. `pm.test("Status code is 200", function () {`
5. `pm.response.to.have.status(200);`
6. `});`
7. `pm.test("Body has name", function () {`
8. `const json = pm.response.json();`
9. `pm.expect(json).to.have.property("name");`
10. `});`

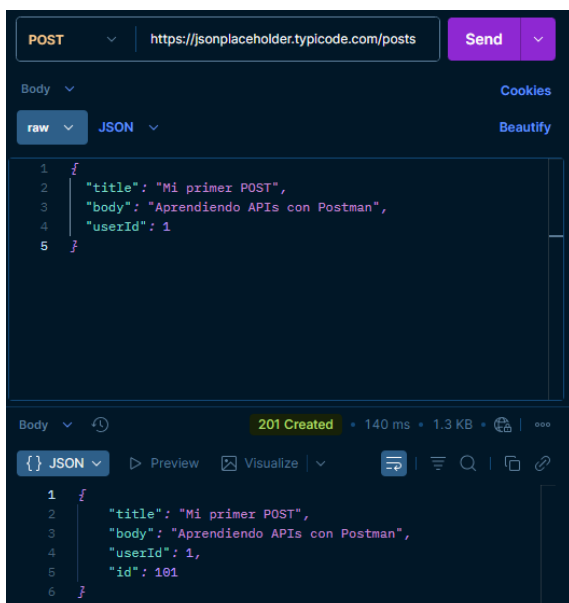
📷 **Entregable:** Captura de los tests en verde.



✿ Ejercicio 8 — POST, PUT y DELETE (API de pruebas)

Usa esta API de prueba:

<https://jsonplaceholder.typicode.com/posts>



POST

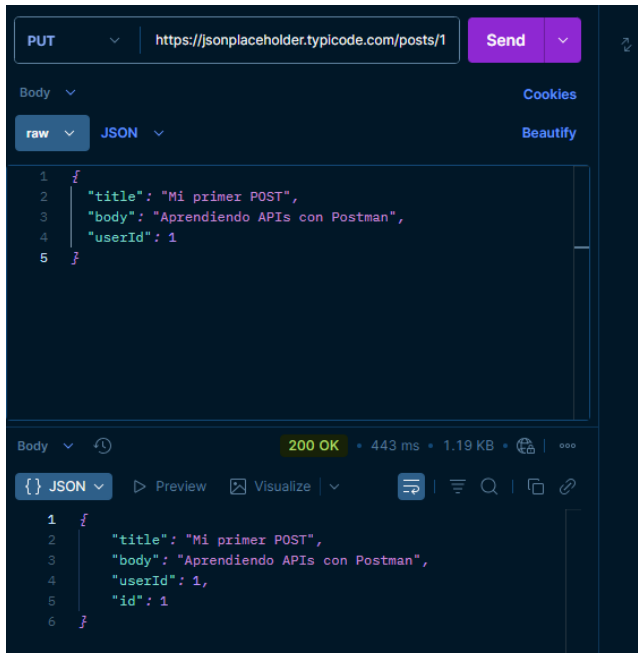
El método **POST** se utiliza para **crear un nuevo recurso** en el servidor[cite: 83].

Al enviar el cuerpo JSON, el servidor de prueba acepta los datos y responde con el código **201 Created**, confirmando que la creación fue exitosa. Además, el servidor devuelve el objeto que enviaste, pero con un campo adicional: "id": 101, que es el **identificador único** asignado al nuevo recurso creado.

PUT

Actualiza un recurso:

<https://jsonplaceholder.typicode.com/posts/1>

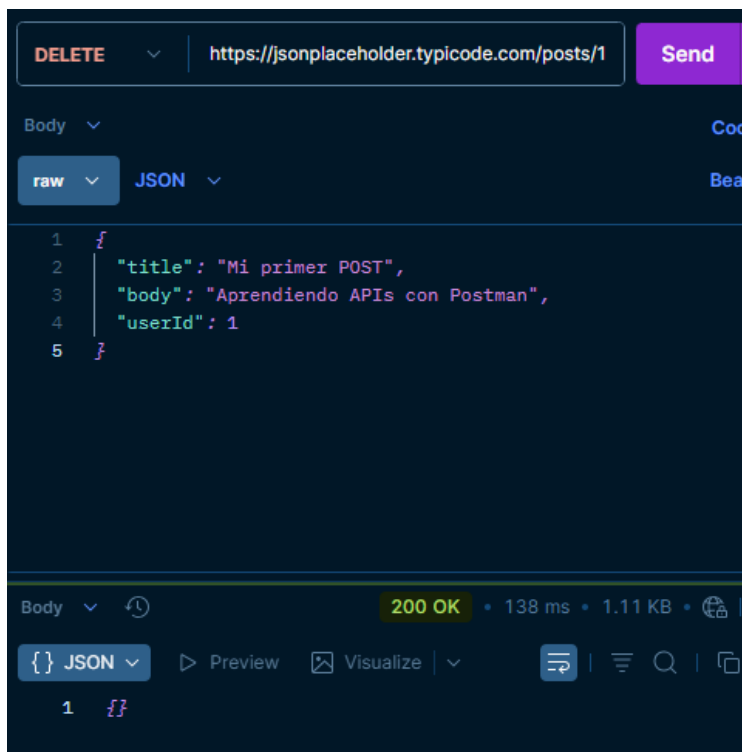


El método **PUT** se utiliza para **actualizar completamente o reemplazar** el recurso ubicado en una URL específica (en este caso, el recurso con id=1)[cite: 89, 91].

El servidor responde con el código **200 OK**, indicando que la solicitud de actualización se procesó con éxito. El cuerpo de la respuesta muestra los datos enviados, confirmando que el recurso con id: 1 ha sido modificado.

DELETE

El método **DELETE** se utiliza para **eliminar** el recurso identificado por la URL específica[cite: 92, 93].



El código de estado **200 OK** (o a veces 204 No Content) confirma que la acción de eliminación en el recurso con id=1 se ha realizado con éxito. El cuerpo de la respuesta es un objeto vacío ({}), ya que el contenido ha sido eliminado.

 **Entregable:** Capturas y explicación de cada método.